



FORMERLY KNOWN AS UETR

Lab Sheet-02

BTCS104T

Advanced Python Lab

B.Tech. II Semester (Session 2025-2026)

Objective: To understand the basic concepts of variable, decision and loop control statements etc.

Requirements:

- Text editor or IDE
- Python 3.xx

Assignments:

1. ATM Withdrawal Simulator

Write a Python program to simulate an ATM withdrawal system where the user enters an initial balance and repeatedly withdraws amounts using a loop. The withdrawal operation must be implemented using a function. The program should stop execution using break when the balance becomes zero or negative.

Sample Input:

Enter balance: 5000

Enter withdrawal amount: 1200

Enter withdrawal amount: 3800

Sample Output:

Remaining balance: 3800

Remaining balance: 0

Transaction completed

2. Student Marks Processing

Write a program that accepts marks of multiple students using a loop and calculates the average marks using a function. If the entered marks are less than 0 or greater than 100, the program should skip that entry using continue.

Sample Input:

Enter marks: 85

Enter marks: -10

Enter marks: 75

Enter marks: 0

Sample Output:

Average marks: 80.0



FORMERLY KNOWN AS UETR

3. Recursive Factorial Calculator

Write a Python program to calculate the factorial of a number using recursion. The program should repeatedly accept numbers using a loop and terminate when the user enters -1.

Sample Input:

Enter number: 5

Enter number: 4

Enter number: -1

Sample Output:

Factorial of 5 is 120

Factorial of 4 is 24

4. Menu Driven Calculator

Write a menu-driven Python program that performs arithmetic operations using functions. The menu should repeat using a loop and exit using break. Use pass for any operation not yet implemented.

Sample Input:

1

10 5

4

Sample Output:

Result: 15

Calculator exited

5. Electricity Bill Calculator

Write a program that calculates electricity bill based on number of units consumed. The calculation logic must be written inside a function. The program should process multiple consumers using a loop.

Sample Input:

Enter units: 120

Enter units: 75

Enter units: 0

Sample Output:

Bill amount: 600

Bill amount: 350



FORMERLY KNOWN AS UETR

6. Number Validation System

Write a program that repeatedly accepts numbers using a loop and checks whether the number is even or odd using a function. Negative numbers should be skipped using continue.

Sample Input:

Enter number: 4

Enter number: -3

Enter number: 7

Enter number: 0

Sample Output:

4 is Even

7 is Odd

7. Recursive Sum of Digits

Write a program to find the sum of digits of a number using recursion. The program should accept multiple numbers using a loop.

Sample Input:

Enter number: 123

Enter number: 56

Enter number: 0

Sample Output:

Sum of digits: 6

Sum of digits: 11

8. Banking Deposit System

Write a program that simulates a banking deposit system where deposits are added using a function. The program should allow multiple deposits using a loop and stop using break when the user enters 0.

Sample Input:

Enter deposit amount: 2000

Enter deposit amount: 1500

Enter deposit amount: 0

Sample Output:

Total balance: 3500



FORMERLY KNOWN AS UETR

9. Prime Number Checker

Write a Python program that checks whether a given number is prime using a function. The program should test multiple numbers using a loop and skip non-positive numbers using continue.

Sample Input:

Enter number: 7

Enter number: -4

Enter number: 10

Enter number: 0

Sample Output:

7 is Prime

10 is Not Prime

10. Student Attendance Counter

Write a program to calculate attendance percentage where attendance is entered as 1 for present and 0 for absent. Use a loop to collect attendance and a function to calculate percentage. Use pass for reserved roll numbers.

Sample Input:

Enter attendance: 1

Enter attendance: 0

Enter attendance: 1

Enter attendance: -1

Sample Output:

Attendance percentage: 66.67%

11. Recursive Fibonacci Series

Write a program that generates Fibonacci numbers using recursion and prints the first N terms using a loop.

Sample Input:

Enter N: 5

Sample Output:

0 1 1 2 3



FORMERLY KNOWN AS UETR

12. Shopping Cart Total

Write a program that accepts item prices using a loop and calculates the total bill using a function. The program should stop accepting inputs when the user enters 0.

Sample Input:

Enter price: 250

Enter price: 450

Enter price: 0

Sample Output:

Total bill: 700

13. Number Guessing Game

Write a Python program where the user guesses a predefined number. Use a loop for repeated attempts and terminate the game using break when the guess is correct.

Sample Input:

Enter guess: 3

Enter guess: 7

Sample Output:

Wrong guess

Correct guess

14. Voting Eligibility System

Write a program that checks voting eligibility using a function. The program should process multiple users using a loop and skip invalid ages using continue.

Sample Input:

Enter age: 20

Enter age: -5

Enter age: 16

Enter age: 0

Sample Output:

Eligible to vote

Not eligible to vote



FORMERLY KNOWN AS UETR

15. Recursive Power Calculator

Write a Python program to calculate power of a number using recursion. The program should test multiple values using a loop.

Sample Input:

Enter base and power: 2 3

Enter base and power: 5 2

Enter 0 0

Sample Output:

Result: 8

Result: 25

16. Multiplication Table Generator

Write a program that generates multiplication tables using a function and nested loops. Multiples of 5 should be skipped using continue.

Sample Input:

Enter number: 4

Sample Output:

4 x 1 = 4

4 x 2 = 8

4 x 3 = 12

4 x 4 = 16

4 x 6 = 24

17. Online Exam Score Calculator

Write a program that accepts marks for multiple questions using a loop and calculates total score using a function. Use pass for unanswered questions.

Sample Input:

Enter marks: 5

Enter marks: -1

Enter marks: 4

Enter marks: 0

Sample Output:

Total score: 9



FORMERLY KNOWN AS UETR

18. Bank Interest Calculator

Write a program that calculates simple interest using a function. The program should process multiple customers using a loop.

Sample Input:

Enter principal, rate, time: 1000 5 2

Enter principal, rate, time: 2000 4 1

Enter 0 0 0

Sample Output:

Interest: 100

Interest: 80

19. Recursive Reverse Number

Write a Python program to reverse a number using recursion and test it for multiple inputs using a loop.

Sample Input:

Enter number: 123

Enter number: 405

Enter number: 0

Sample Output:

Reversed number: 321

Reversed number: 504

20. Salary Slip Generator

Write a program that calculates total salary using a function by adding basic pay and bonus. The program should process multiple employees using a loop.

Sample Input:

Enter basic and bonus: 20000 5000

Enter basic and bonus: 15000 3000

Enter 0 0

Sample Output:

Total salary: 25000

Total salary: 18000



FORMERLY KNOWN AS UETR

21. Number Frequency Counter

Write a Python program that accepts numbers using a loop and counts how many times a given number appears. The counting logic must be written using a function.

Sample Input:

Enter numbers: 2 4 2 5 2 0

Enter number to search: 2

Sample Output:

Frequency of 2 is 3

22. Mini Calculator Using Module

Write a Python program that creates a user-defined module containing arithmetic functions. Import the module in the main program and perform calculations repeatedly using a loop until the user chooses to exit using break.

Sample Input:

1

10 5

4

Sample Output:

Result: 15

Program terminated

23. Recursive GCD Calculator

Write a Python program to calculate the Greatest Common Divisor of two numbers using recursion. The program should allow multiple test cases using a loop.

Sample Input:

Enter numbers: 24 36

Enter numbers: 10 15

Enter 0 0

Sample Output:

GCD is 12

GCD is 5



FORMERLY KNOWN AS UETR

24. Bus Ticket Booking System

Write a program to simulate a bus ticket booking system where available seats are reduced after each booking. Use a function to update seats, a loop for repeated booking, and break when seats are full.

Sample Input:

Enter seats to book: 3

Enter seats to book: 2

Enter seats to book: 0

Sample Output:

Seats remaining: 7

Seats remaining: 5

25. Temperature Analysis System

Write a program that accepts temperature values using a loop and finds the maximum and minimum temperature using a function. Invalid temperatures should be skipped using continue.

Sample Input:

Enter temperature: 30

Enter temperature: -5

Enter temperature: 25

Enter temperature: 0

Sample Output:

Maximum temperature: 30

Minimum temperature: 25

26. Exam Result Analyzer

Write a Python program that accepts marks of students using a loop and assigns grades using a function. Use pass for students marked absent.

Sample Input:

Enter marks: 85

Enter marks: -1

Enter marks: 40

Enter marks: 0

Sample Output:

Grade: A

Grade: D



27. Recursive Digit Counter

Write a program to count the number of digits in a number using recursion. The program should test multiple numbers using a loop.

Sample Input:

Enter number: 4567

Enter number: 90

Enter number: 0

Sample Output:

Number of digits: 4

Number of digits: 2

28. Daily Expense Tracker

Write a program that tracks daily expenses using a loop and calculates total expense using a function. The program should stop when the user enters 0 using break.

Sample Input:

Enter expense: 200

Enter expense: 350

Enter expense: 0

Sample Output:

Total expense: 550

29. Number Pattern Generator

Write a Python program that generates a number pattern using nested loops. Use a function to generate the pattern and continue to skip specific rows.

Sample Input:

Enter number of rows: 4

Sample Output:

1

12

123

1234

30. Mini Utility Program

Write a Python program that demonstrates the use of multiple functions, loops, recursion, and loop control statements such as break, continue, and pass in a single application.

• Solutions:

```
• import mymath
• # question 1
• def withdrawalsimulator(balance,amount):
•     balance=balance-amount
•     return balance
• balance=int(input("Enter initial balance:"))
• while True:
•     amount=int(input("Enter withdrawal amount:"))
•     balance=withdrawalsimulator(balance,amount)
•     print("Remaining balance:",balance)
•     if balance<=0:
•         print("Transaction Completed:")
•         break
•
• #question 2
• def MarksProcessing(total_marks,count):
•     if count==0:
•         return 0
•     return total_marks/count
• total_marks=0
• count=0
• while True:
•     marks=int(input("Enter Marks:"))
•     if marks==0:
•         break
•     if marks < 0 or marks > 100:
•         continue
•     total_marks+=marks
•     count+=1
• average=MarksProcessing(total_marks,count)
• print("Average:",average)
•
• #question 3
• def factorial(number):
•     if number==0 or number==1:
•         return 1
•     return number*factorial(number-1)
• while True:
•     number=int(input("Enter a number:"))
•     if number==-1:
•         print("Exit")
•         break
•     fact=factorial(number)
•     print(f"Factorial of {number} is {fact}")
•
• #question 4
```

```

• def add(a, b):
•     return a + b
•
• def subtract(a, b):
•     return a - b
•
• def multiply(a, b):
•     return a * b
•
• def divide(a, b):
•     if b == 0:
•         return "Cannot divide by zero"
•     return a / b
•
• while True:
•     print("\n--- Calculator Menu ---")
•     print("1. Addition")
•     print("2. Subtraction")
•     print("3. Multiplication")
•     print("4. Exit")
•     print("5. Division (Not implemented)")
•
•     choice = int(input("Enter choice: "))
•
•     if choice == 4:
•         print("Calculator exited")
•         break
•
•     if choice == 5:
•         pass
•         continue
•
•     a, b = map(int, input("Enter two numbers: ").split())
•
•     if choice == 1:
•         print("Result:", add(a, b))
•     elif choice == 2:
•         print("Result:", subtract(a, b))
•     elif choice == 3:
•         print("Result:", multiply(a, b))
•     else:
•         print("Invalid choice")
•
• #question 5
• def Elcetricitybill(units):
•     if units<=100:
•         return units*5

```

```

•     else:
•         return units*6
• while True:
•     units=int(input("Enter units:"))
•     if units==0:
•         break
•     if units<0:
•         continue
•     Bill=Elcectricitybill(units)
•     print("Electricity bill :",Bill)
•
• #question 6
• def even_odd(num):
•     if num%2==0:
•         print("Even number")
•     else:
•         print("Odd number")
• while True:
•     num=int(input("Enter a number:"))
•     if num<0:
•         continue
•     if num==0:
•         break
•     even_odd(num)
•
• #question 7
• def sum_of_digit(n):
•     if n==0:
•         return 0
•     return n%10+sum_of_digit(n//10)
• while True:
•     n=int(input("Enter number:"))
•     if n==0:
•         break
•     sum=sum_of_digit(n)
•     print(f"Sum of digit of {n} is {sum}")
•
• # question 8
• def banking_deposit(amount,balance):
•     balance+=amount
•     return balance
• balance=0
• while True:
•     Deposit_amount=int(input("Enter deposited Amount:"))
•     if Deposit_amount==0:
•         break
•     balance=banking_deposit(Deposit_amount,balance)

```

```

• print("Total balance:",balance)
•
• #question 9
• def is_prime(num):
•     if num<=1:
•         return False
•     for i in range(2,int(num**0.5)+1):
•         if num%i==0:
•             return False
•     return True
• while True:
•     num=int(input("Enter number:"))
•     if num<0:
•         continue
•     if num==0:
•         break
•     if is_prime(num):
•         print(f"{num} is Prime:")
•     else:
•         print(f"{num} is Not Prime")
•
• #question 10
• def attendance_percentage(present,total_attendance):
•     return (present/total_attendance*100)
• present=0
• total=0
• while True:
•     attendance=int(input("Enter attendance"))
•     if attendance==-1:
•         break
•     elif attendance==1:
•         present+=1
•         total+=1
•     elif attendance==0:
•         total+=1
•     else:
•         pass
• if total>0:
•     percenatage=attendance_percentage(present,total)
•     print(f"Attendace percentage {percenatage:.2f}%")
•
• #question 11
• def fibonacci_series(n):
•     if n==0:
•         return 0
•     elif n==1:
•         return 1
•     return fibonacci_series(n-1)+fibonacci_series(n-2)

```

```

• n=int(input("Enter N"))
• for i in range(n):
•     result=fibonacci_series(i)
•     print(result,end=" ")
•
• # question 12
• def shoppingcart(total):
•     return total
• total=0
• while True:
•     price=int(input("Enter Price:"))
•     if price==0:
•         break
•     else:
•         total+=price
• total_bill=shoppingcart(total)
• print("Total bill=",total_bill)
•
• #question 13
• def guessgame():
•     number = 7          # predefined number
•     attempts = 0
•
•     print("=== Number Guessing Game ===")
•
•     while True:
•         guess = int(input("Enter guess: "))
•         attempts += 1
•
•         if guess == number:
•             print("Correct guess")
•             print("Total attempts =", attempts)
•             break
•         elif guess > number:
•             print("Too High! Try again")
•         else:
•             print("Too Low! Try again")
•
• guessgame()
•
• #question 14
• def voting_eligiblity(age):
•     if age<18:
•         return False
•     else:
•         return True
• while True:
•     age=int(input("Enter age:"))

```

```

•     if age<0:
•         continue
•     if age==0:
•         break
•     if voting_eligiblity(age):
•         print("Eligible to vote")
•     else:
•         print("Not Eligible to vote ")
•
•
• question 15
• def power_calculator(base,power):
•     if power==0:
•         return 1
•     return base*power_calculator(base,power-1)
• while True:
•     base,power=map(int,input("Enter base and power:").split())
•     if base<0:
•         continue
•     if base==0:
•         break
•     result=power_calculator(base,power)
•     print("Result:",result)
•
• #question 16
• def table_generator():
•     n=int(input("Enter number:"))
•     for i in range(1,11):
•         result=n*i
•         if result%5==0:
•             continue
•         print(n,"X",i, "=",result)
• table_generator()
•
• question 17
• def Score_calculator(total):
•     return total
• total=0
• while True:
•     marks=int(input("Enter marks:"))
•     if marks==0:
•         break
•     elif marks==-1:
•         pass
•     total+=marks
• result=Score_calculator(total)
• print("Total score:",result)
•
• #question 18

```

```

• def SI(p,r,t):
•     return (p*r*t)/100
• while True:
•     p,r,t=map(int,input("Enter principal,rate,time:").split())
•     if p==0 and r==0 and t==0:
•         break
•     Simple_intrest=SI(p,r,t)
•     print("Interest:",Simple_intrest)
•
• question 19
•
• def reverse_number(n,rev=0):
•     if n==0:
•         return rev
•     return(reverse_number(n//10,rev*10+n%10))
• while True:
•     n=int(input("Enter number:"))
•     if n==0:
•         break
•     reverse=reverse_number(n)
•     print("Reversed number:",reverse)
•
• #question 20
•
• def Salary(basic,bonus):
•     return(basic+bonus)
• while True:
•     basic,bonus=map(int,input("Enter basic and bonus:").split())
•     if basic==0 and bonus==0:
•         break
•     salary_Slip=Salary(basic,bonus)
•     print("Total Salary:",salary_Slip)
•
• # # # question 21
•
• # def frequency_counter(number,key):
• #     count=0
• #     for num in numbers:
• #         if num==key:
• #             count+=1
• #     return count
• # while True:
• #     numbers=list(map(int,input("Enter numbers:").split()))
• #     key=int(input("Enter number to searh:"))
• #     result=frequency_counter(numbers,key)
• #     print("Frequency of ",key," is ",result)
•

```

```

• #question 22
•
• while True:
•     choice = int(input("Enter choice: "))
•
•     if choice == 4:
•         print("Calculator exited")
•         break
•
•     if choice == 5:
•         pass
•         continue
•
•     a, b = map(int, input("Enter two numbers: ").split())
•
•     if choice == 1:
•         print("Result:", mymath.add(a, b))
•     elif choice == 2:
•         print("Result:", mymath.subtract(a, b))
•     elif choice == 3:
•         print("Result:", mymath.multiply(a, b))
•     else:
•         print("Invalid choice")
•
• #question 23
• def gcd(a, b):
•     if b == 0:
•         return a
•     return gcd(b, a % b)
•
• while True:
•     a, b = map(int, input("Enter numbers: ").split())
•
•     if a == 0 and b == 0:
•         break
•
•     result = gcd(a, b)
•     print("GCD is", result)
•
• # question 24
• def bus_ticket(book,total):
•     total=total-book
•     return total
• total=30
• while True:
•     book=int(input("Enter seats to book:"))

```

```

•     if book==0 or total==0:
•         break
•
•     total=bus_ticket(book,total)
•     print("Seats remaining:",total)
•
• #question 25
• def find_max_min(temps):
•     return max(temps), min(temps)
•
• temperatures = []
•
• while True:
•     t = int(input("Enter temperature: "))
•
•     if t == 0:
•         break
•
•     if t < 0:
•         continue
•
•     temperatures.append(t)
•
•
• maximum, minimum = find_max_min(temperatures)
•
• print("Maximum temperature:", maximum)
• print("Minimum temperature:", minimum)
•
• #question 26
• def give_grade(marks):
•     if marks >= 80:
•         return "A"
•     elif marks >= 60:
•         return "B"
•     elif marks >= 50:
•         return "C"
•     else:
•         return "D"
•
• while True:
•     marks = int(input("Enter marks: "))
•
•     if marks == 0:
•         break
•
•     if marks == -1:
•         pass

```

```

•         continue
•
•         print("Grade:", give_grade(marks))
•
• #question 27
• def count_digits(n):
•     if n < 10:
•         return 1
•     return 1 + count_digits(n // 10)
•
• while True:
•     n = int(input("Enter number: "))
•     if n == 0:
•         break
•
•     print("Number of digits:", count_digits(abs(n)))
•
• #question 28
• def total_expense(expenses):
•     return sum(expenses)
•
• expenses = []
•
• while True:
•     amt = int(input("Enter expense: "))
•     if amt == 0:
•         break
•     expenses.append(amt)
•
• print("Total expense:", total_expense(expenses))
•
• #question 29
• def pattern(rows):
•     for i in range(1, rows + 1):
•         if i == 0:      # example skip condition
•             continue
•         for j in range(1, i + 1):
•             print(j, end="")
•         print()
•
• rows = int(input("Enter number of rows: "))
• pattern(rows)
•
• #question 30
• def factorial(n):
•     if n == 0 or n == 1:
•         return 1
•     return n * factorial(n - 1)

```



```
def add(a, b):  
    return a + b  
  
while True:  
    print("\n1.Add 2.Factorial 3.Skip 4.Exit")  
    choice = int(input("Enter choice: "))  
  
    if choice == 4:  
        print("Program ended")  
        break  
  
    if choice == 3:  
        pass  
        continue  
  
    if choice == 1:  
        a, b = map(int, input("Enter numbers: ").split())  
        print("Result:", add(a, b))  
  
    elif choice == 2:  
        n = int(input("Enter number: "))  
        print("Factorial:", factorial(n))  
  
    else:  
        print("Invalid choice")
```